

Gesture Detector

13 Gesture Code Reference

Generated from the current `gesture_detector` codebase on 2026-07-04. This guide documents how each gesture is used in code, which landmarks are read, and which thresholds/angles matter.

Live Detection Priority

- A camera frame is processed only every 100 ms or more.
- Hands below `minHandScore 0.45` are ignored for main gesture actions.
- The app first evaluates cancel/OK/call/punch custom gestures and follow-object state.
- Recording holds are processed before zoom and movement.
- Zoom gestures run before generic movement so pinch gestures do not become Moving up.
- Known package gestures such as `thumbUp` and `victory` block generic movement.
- Generic direction movement is the fallback action.

Common Geometry Helpers

- `handSize = max(hand.boundingBox.width, hand.boundingBox.height)`.
- `palmCenter` = average of `wrist`, `indexMCP`, `middleMCP`, `ringMCP`, and `pinkyMCP` when available.
- `fingerJointAngleDegrees(mcp, pip, tip)` measures the PIP angle for non-thumb fingers.
- Visible landmarks require `visibility >= 0.35`, while zoom uses `>= 0.30`.
- Image Y grows downward, so upward motion has negative `deltaY`.

Landmark Point Map Used By This Code

The code uses HandLandmarkType names. The numeric ids below follow the common 21-point hand landmark layout. This project directly references the listed names; id 1 is included only for orientation.

ID	Code point	Where used
0	wrist	Palm center, palm side checks, open palm orientation.
1	thumbCMC	Standard 21-point id; not referenced by this repo.
2	thumbMCP	Thumb angle, closed thumb, open palm thumb score.
3	thumbIP	Thumb angle, call gesture, closed thumb checks.
4	thumbTip	OK touch, call gesture, zoom, open palm, closed thumb.
5	indexMCP	Direction chain, index angle, circle gesture, palm center.
6	indexPIP	Direction chain, index angle, folded/open checks.
7	indexDIP	Direction chain and index circle direction point.
8	indexTip	Direction chain, OK touch, circle tracking, zoom.
9	middleMCP	Direction chain, palm center, open/folded checks.
10	middlePIP	Direction chain and angle/fold checks.
11	middleDIP	Direction chain.
12	middleTip	Direction chain and open/folded checks.
13	ringMCP	Direction chain, palm center, open/folded checks.
14	ringPIP	Direction chain and angle/fold checks.
15	ringDIP	Direction chain.
16	ringTip	Direction chain and open/folded checks.
17	pinkyMCP	Direction chain, palm center, open/folded checks.
18	pinkyPIP	Direction chain and angle/fold checks.
19	pinkyDIP	Direction chain.
20	pinkyTip	Direction chain, call gesture, open/folded checks.

1. Move Left

Move or hold an extended hand/finger shape toward the left. The app reports Moving left.

Code path	• DirectionGestureDetector.detect(...) in lib/hand_gesture_features/domain/services/direction_gesture_detector.dart. • Called from _updateGestureState after custom gestures, recording, zoom, and known package gestures have not won.
How to use in code	• Get bestHand from the camera hand detector. • Call _directionGestureDetector.detect(hand: bestHand, imageSize: detectionImageSize, mirrorHorizontally: mirrorDirectionalGestureCoordinates). • If result == HandMoveDirection.left, show Moving left or send your stand-left command.

Points used	• Index chain: indexFingerMCP -> indexFingerPIP -> indexFingerDIP -> indexFingerTip (ids 5, 6, 7, 8). • Middle chain: middleFingerMCP -> middleFingerPIP -> middleFingerDIP -> middleFingerTip (ids 9, 10, 11, 12). • Ring chain: ringFingerMCP -> ringFingerPIP -> ringFingerDIP -> ringFingerTip (ids 13, 14, 15, 16). • Pinky chain: pinkyMCP -> pinkyPIP -> pinkyDIP -> pinkyTip (ids 17, 18, 19, 20). • X is optionally mirrored before direction math; Y increases downward in image coordinates.
Angles and thresholds	• A finger chain counts only when angle MCP-PIP-TIP is ≥ 160 deg. • At least 3 accepted finger chains must agree on the same axis. • Horizontal minimum distance = $\max(\text{imageWidth} * 0.025, \text{fingerSpan} * 0.18)$. • Vertical minimum distance = $\max(\text{imageHeight} * 0.025, \text{fingerSpan} * 0.18)$. • Horizontal dominance: $\text{abs}(\text{deltaX}) \geq \text{abs}(\text{deltaY}) * 0.45$. • Vertical dominance: $\text{abs}(\text{deltaY}) \geq \text{abs}(\text{deltaX}) * 0.45$. • Left condition: summed chain $\text{deltaX} < 0$ after mirroring, and horizontal rules pass.
Overlap priority	• Direction is a fallback. It is skipped while follow-object, recording, zoom, or a known package gesture is active. • Folded fingers no longer count as direction chains, which prevents zoom and fist poses from becoming left movement.

2. Move Right

Move or hold an extended hand/finger shape toward the right. The app reports Moving right.

Code path	• <code>DirectionGestureDetector.detect(...)</code> in <code>direction_gesture_detector.dart</code>. • The same four finger chains are checked as Move Left, but the X direction is positive.
How to use in code	• Call <code>DirectionGestureDetector.detect</code> with the selected hand and detection image size. • If <code>result == HandMoveDirection.right</code>, show Moving right or send your stand-right command. • Keep <code>mirrorHorizontally</code> aligned with the preview/camera direction so left and right are user-visible directions.
Points used	• Index chain: <code>indexFingerMCP</code> -> <code>indexFingerPIP</code> -> <code>indexFingerDIP</code> -> <code>indexFingerTip</code> (ids 5, 6, 7, 8). • Middle chain: <code>middleFingerMCP</code> -> <code>middleFingerPIP</code> -> <code>middleFingerDIP</code> -> <code>middleFingerTip</code> (ids 9, 10, 11, 12). • Ring chain: <code>ringFingerMCP</code> -> <code>ringFingerPIP</code> -> <code>ringFingerDIP</code> -> <code>ringFingerTip</code> (ids 13, 14, 15, 16). • Pinky chain: <code>pinkyMCP</code> -> <code>pinkyPIP</code> -> <code>pinkyDIP</code> -> <code>pinkyTip</code> (ids 17, 18, 19, 20). • X is optionally mirrored before direction math; Y increases downward in image coordinates.
Angles and thresholds	• A finger chain counts only when angle MCP-PIP-TIP is ≥ 160 deg. • At least 3 accepted finger chains must agree on the same axis. • Horizontal minimum distance = $\max(\text{imageWidth} * 0.025, \text{fingerSpan} * 0.18)$. • Vertical minimum distance = $\max(\text{imageHeight} * 0.025, \text{fingerSpan} * 0.18)$. • Horizontal dominance: $\text{abs}(\text{deltaX}) \geq \text{abs}(\text{deltaY}) * 0.45$. • Vertical dominance: $\text{abs}(\text{deltaY}) \geq \text{abs}(\text{deltaX}) * 0.45$. • Right condition: summed chain <code>deltaX</code> > 0 after mirroring, and horizontal rules pass.
Overlap priority	• Direction is checked only after higher-priority gestures are rejected. • This stops thumbs-up and zoom from being displayed as movement.

3. Move Up

Point or move an extended hand/finger shape upward. The app reports Moving up.

Code path	• <code>DirectionGestureDetector.detect(...)</code> in <code>direction_gesture_detector.dart</code>. • README also documents pointing up as a Move Up input, but package <code>pointingUp</code> can also be shown as its own known gesture.
How to use in code	• Call <code>DirectionGestureDetector.detect</code> after zoom and package gestures are checked. • If <code>result == HandMoveDirection.up</code>, show Moving up or send your stand-up command.
Points used	• Index chain: <code>indexFingerMCP</code> -> <code>indexFingerPIP</code> -> <code>indexFingerDIP</code> -> <code>indexFingerTip</code> (ids 5, 6, 7, 8). • Middle chain: <code>middleFingerMCP</code> -> <code>middleFingerPIP</code> -> <code>middleFingerDIP</code> -> <code>middleFingerTip</code> (ids 9, 10, 11, 12). • Ring chain: <code>ringFingerMCP</code> -> <code>ringFingerPIP</code> -> <code>ringFingerDIP</code> -> <code>ringFingerTip</code> (ids 13, 14, 15, 16). • Pinky chain: <code>pinkyMCP</code> -> <code>pinkyPIP</code> -> <code>pinkyDIP</code> -> <code>pinkyTip</code> (ids 17, 18, 19, 20). • X is optionally mirrored before direction math; Y increases downward in image coordinates.
Angles and thresholds	• A finger chain counts only when angle MCP-PIP-TIP is ≥ 160 deg. • At least 3 accepted finger chains must agree on the same axis. • Horizontal minimum distance = $\max(\text{imageWidth} * 0.025, \text{fingerSpan} * 0.18)$. • Vertical minimum distance = $\max(\text{imageHeight} * 0.025, \text{fingerSpan} * 0.18)$. • Horizontal dominance: $\text{abs}(\text{deltaX}) \geq \text{abs}(\text{deltaY}) * 0.45$. • Vertical dominance: $\text{abs}(\text{deltaY}) \geq \text{abs}(\text{deltaX}) * 0.45$. • Up condition: summed chain $\text{deltaY} < 0$, because image Y gets smaller upward.
Overlap priority	• Zoom is evaluated before movement. If a zoom pose is active, Move Up is skipped. • Folded upward fingers are rejected by the ≥ 160 deg extension gate.

4. Move Down

Point or move an extended hand/finger shape downward. The app reports Moving down.

Code path	• <code>DirectionGestureDetector.detect(...)</code> in <code>direction_gesture_detector.dart</code>. • The same chain logic as Move Up is used, but the Y direction is positive.
How to use in code	• Call <code>DirectionGestureDetector.detect</code> from the live gesture pipeline. • If <code>result == HandMoveDirection.down</code>, show Moving down or send your stand-down command.
Points used	• Index chain: <code>indexFingerMCP</code> -> <code>indexFingerPIP</code> -> <code>indexFingerDIP</code> -> <code>indexFingerTip</code> (ids 5, 6, 7, 8). • Middle chain: <code>middleFingerMCP</code> -> <code>middleFingerPIP</code> -> <code>middleFingerDIP</code> -> <code>middleFingerTip</code> (ids 9, 10, 11, 12). • Ring chain: <code>ringFingerMCP</code> -> <code>ringFingerPIP</code> -> <code>ringFingerDIP</code> -> <code>ringFingerTip</code> (ids 13, 14, 15, 16). • Pinky chain: <code>pinkyMCP</code> -> <code>pinkyPIP</code> -> <code>pinkyDIP</code> -> <code>pinkyTip</code> (ids 17, 18, 19, 20). • X is optionally mirrored before direction math; Y increases downward in image coordinates.
Angles and thresholds	• A finger chain counts only when angle MCP-PIP-TIP is ≥ 160 deg. • At least 3 accepted finger chains must agree on the same axis. • Horizontal minimum distance = $\max(\text{imageWidth} * 0.025, \text{fingerSpan} * 0.18)$. • Vertical minimum distance = $\max(\text{imageHeight} * 0.025, \text{fingerSpan} * 0.18)$. • Horizontal dominance: $\text{abs}(\text{deltaX}) \geq \text{abs}(\text{deltaY}) * 0.45$. • Vertical dominance: $\text{abs}(\text{deltaY}) \geq \text{abs}(\text{deltaX}) * 0.45$. • Down condition: summed chain $\text{deltaY} > 0$, because image Y gets larger downward.
Overlap priority	• Movement is ignored when follow tracking, custom gestures, recording, zoom, or known package gestures are active.

5. Detect My Face

Hold the call-me hand shape for 2 seconds. The app then searches for the best face target.

Code path	• CustomGestureDetector._isCallMeGesture(...) sets rawCustomGestureResult.isCallMe. • _updateGestureState holds _faceDetectGestureStartedAt until faceDetectHoldDuration is reached, then calls _selectBestFaceTarget(...).
How to use in code	• Call _customGestureDetector.detect(...) for the selected hand. • If result.isCallMe is true and no follow target is already locked, start/continue a 2 second hold timer. • When hold progress reaches 1.0, call face detection and lock the best face target.
Points used	• Thumb: thumbTip and thumbIP (ids 4, 3). • Pinky: pinkyTip and pinkyPIP (ids 20, 18). • Closed fingers: indexTip/indexPIP (8, 6), middleTip/middlePIP (12, 10), ringTip/ringPIP (16, 14). • Palm center: average of wrist plus index/middle/ring/pinky MCP points.
Angles and thresholds	• Thumb open: $\text{dist}(\text{thumbTip}, \text{palmCenter}) > \text{dist}(\text{thumbIP}, \text{palmCenter}) * 1.15$. • Thumb reach: $\text{dist}(\text{thumbTip}, \text{palmCenter}) > \text{handSize} * 0.23$. • Pinky open: $\text{tip distance} > \text{pip distance} * 1.20$ and $\text{tip distance} > \text{handSize} * 0.30$. • Index, middle, ring closed: $\text{tip distance} \leq \text{pip distance} * 1.03$ OR $\text{tip distance} < \text{handSize} * 0.26$. • Thumb and pinky separation: $\text{dist}(\text{thumbTip}, \text{pinkyTip}) > \text{handSize} * 0.55$. • Face detect hold: 2 seconds.
Overlap priority	• This custom gesture runs before movement and zoom display, so it should not fall through to Moving left/up.

6. Follow The Object

Open palm, hold, closed fist, then open palm again. The final hand-box center is used as the release point.

Code path	• FollowObjectSequenceDetector.update(...) controls the sequence. • Open palm is custom geometry via OpenPalmGestureDetector. Closed fist in this sequence is package GestureType.closedFist. • _selectFollowTargetAtReleasePoint(...) selects a face or object at release.
How to use in code	• Show open palm and hold for followObjectFirstOpenPalmHoldDuration (1 second). • Show closed fist; this moves the sequence to waitingForFinalOpen. • Move the hand over the object/face and open palm again. • Use releasePoint = center of hand.boundingBox to choose the target.
Points used	• Open palm uses wrist plus thumbMCP/thumbIP/thumbTip and index/middle/ring/pinky MCP/PIP/tip points. • Open palm also uses indexTip, middleTip, ringTip, pinkyTip spread distances. • Closed fist for this sequence is package-classified; this repo does not define its sequence points. • Release point is not a landmark; it is the center of the detected hand bounding box.
Angles and thresholds	• Open palm enter confidence: 0.55; exit confidence: 0.45. • Smoothing: 4 samples, at least 2 positive samples, max sample age 500 ms. • Four fingers score angle 145..165 deg, tip/pip distance ratio 1.08..1.22, reach 0.25..0.34 of handSize. • Thumb score angle 125..155 deg plus thumb reach and separation from index. • Spread score uses indexTip to pinkyTip, adjacent tip distances, and indexMCP to pinkyMCP fan ratio. • Closed fist package confidence: >= 0.50 for the follow sequence.
Overlap priority	• While the follow sequence is active, custom gestures, recording, zoom, and movement are suppressed.

7. Stop and Continue Action

Show thumbs-up. The current code displays Stop & Continue Action for a package thumbUp gesture.

Code path	• Package classifier result: <code>hand.gesture.type == GestureType.thumbUp</code>. • Handled in <code>_updateGestureState</code> under <code>hasKnownGesture</code>.
How to use in code	• Read <code>bestHand.gesture</code> after follow tracking and custom gestures are handled. • Require <code>gesture.type == GestureType.thumbUp</code> and <code>confidence >= minPackageGestureConfidence (0.50)</code>. • Set gesture text or run your stop/continue command. • README says hold 1 second; current code does not yet add a thumbUp hold timer.
Points used	• No direct landmark math for thumbUp exists in this repo. • The <code>hand_detection</code> package decides thumbUp internally and returns <code>GestureType.thumbUp</code>. • If you want local geometry later, use <code>thumbTip/thumbIP/thumbMCP</code> and folded <code>index/middle/ring/pinky</code> checks.
Angles and thresholds	• Package gesture confidence: <code>>= 0.50</code>. • No repo-defined angle thresholds for thumbUp.
Overlap priority	• Known package gestures now block generic movement, so thumbUp should not show Moving left.

8. Return To Main Position

Rotate the index finger in a small circle while only the index finger is extended upward.

Code path	• CustomGestureDetector._detectCancelEverythingGesture(...). • Result label: Return to main position. In the live screen this clears active tasks and resets camera zoom.
How to use in code	• Call _customGestureDetector.detect(...) every processed frame. • When result.isCancelEverything is true, call _clearAllActiveGestureTasks(resetCameraZoom: true). • Keep passing mirrorHorizontally so index-tip circle points match visible direction.
Points used	• Index-only pose uses thumbTip/thumbIP/thumbMCP, indexMCP/indexPIP/indexDIP/indexTip, middle/ring/pinky MCP/PIP/tip. • Circle history tracks only visible indexFingerTip (id 8). • Palm center uses wrist and four MCP points.
Angles and thresholds	• Index extended: MCP-PIP-TIP angle ≥ 160 deg and tip distance $> \text{handSize} * 0.30$. • Index faces up: indexTip.y $< \text{indexPIP.y}$ and indexTip.y $< \text{palmCenter.y} - \text{handSize} * 0.08$. • Index upright side offset: $\text{abs}(\text{indexTip.x} - \text{indexMCP.x}) \leq \text{handSize} * 0.50$. • Thumb closed: thumbTip to palm $\leq 0.30 * \text{handSize}$; thumbTip to IP ratio ≤ 1.00. • Thumb close to knuckles: $\min(\text{thumbTip-indexMCP}, \text{thumbTip-middleMCP}) \leq 0.32 * \text{handSize}$. • Thumb not stretched: $\text{thumbTip-thumbMCP} \leq 0.36 * \text{handSize}$. • Middle, ring, pinky folded by angle: ≤ 145 deg. • Circle history: max 36 samples, 1400 ms window, at least 5 samples. • Circle radius: $\max(\min(\text{imageWidth}, \text{imageHeight}) * 0.006, \text{handSize} * 0.015)$. • Radius variation: $\leq \text{averageRadius} * 1.60$. • Total circular angle: $\text{abs}(\text{totalAngle}) \geq \pi * 0.90$.
Overlap priority	• This custom gesture is handled before recording, zoom, package gestures, and direction movement.

9. Start Record Video

Hold the OK gesture for 1 second. The app starts video recording.

Code path	• CustomGestureDetector._isOkGesture(...) sets customGestureResult.isOk. • recording_controls.dart maps isOk to _RecordingGestureAction.start.
How to use in code	• Call _customGestureDetector.detect(...) and require exactly one custom label. • If result.isOk is true, pass _RecordingGestureAction.start to _updateRecordingGestureHold. • After recordStartHoldDuration reaches 1 second, call _startGestureVideoRecording(controller).
Points used	• Thumb/index touch: thumbTip and indexTip (ids 4, 8). • Index bend: indexMCP, indexPIP, indexTip (5, 6, 8). • Open fingers: middle/ring/pinky MCP/PIP/tip. • Palm center for extension checks.
Angles and thresholds	• Thumb-index touch distance $\leq \max(\text{handSize} * 0.11, 12 \text{ px})$. • Index bend angle MCP-PIP-TIP $\leq 150 \text{ deg}$. • Middle/ring/pinky extended by angle: $\geq 160 \text{ deg}$ and tip distance $> \text{handSize} * 0.30$. • Hold duration: 1 second. • Only allowed when not already recording.
Overlap priority	• Recording gestures run before zoom and movement. • If multiple custom gestures overlap, the app shows Hand detected instead of triggering recording.

10. Pause Video

Make a fist and hold for 1 second. The app toggles pause/resume while recording.

Code path	• CustomGestureDetector._isPunchGesture(...) sets customGestureResult.isPunch. • recording_controls.dart maps isPunch to _RecordingGestureAction.togglePause.
How to use in code	• Call _customGestureDetector.detect(...) and require exactly one custom label. • If result.isPunch is true and video is recording, pass togglePause to _updateRecordingGestureHold. • After 1 second, call pauseVideoRecording() or resumeVideoRecording().
Points used	• Index, middle, ring, pinky tip/PIP/MCP points. • Palm center for folded checks. • Knuckle alignment uses y values of indexMCP, middleMCP, ringMCP, pinkyMCP.
Angles and thresholds	• Each finger folded: tip distance $\leq$ pip distance * 1.03 OR tip distance $<$ handSize * 0.26. • Knuckle Y spread $\leq$ handSize * 0.12. • Package closedFist support can help when confidence $\geq$ 0.60, but fingers still must be closed. • Hold duration: 1 second. • Only allowed while video is already recording.
Overlap priority	• Recording feedback suppresses movement and zoom while the hold is active.

11. End Record Video

Hold the victory gesture for 2 seconds. The app stops and saves the recording.

Code path	• Package classifier result: <code>hand.gesture.type == GestureType.victory</code>. • <code>recording_controls.dart</code> maps <code>hasVictoryGesture</code> to <code>_RecordingGestureAction.stop</code>.
How to use in code	• Ensure follow tracking is not active and no custom gesture label is present. • Require <code>GestureType.victory</code> with confidence <code>>= 0.50</code>. • Pass <code>_RecordingGestureAction.stop</code> to <code>_updateRecordingGestureHold</code>. • After 2 seconds, call <code>_stopGestureVideoRecording(controller)</code>.
Points used	• No direct victory landmark math exists in this repo. • The <code>hand_detection</code> package decides victory internally. • Typical victory geometry would involve index/middle extended and ring/pinky folded, but that is not implemented here.
Angles and thresholds	• Package gesture confidence: <code>>= 0.50</code>. • Hold duration: 2 seconds. • Only allowed while video is recording.
Overlap priority	• Victory is checked before zoom/movement and is used only for recording stop.

12. Zoom In

Start with thumb and index pinched, hold briefly, then open them outward.

Code path	• <code>ZoomGestureDetector.detect(...)</code> state machine. • <code>_handleZoomDirection(ZoomDirection.zoomIn)</code> applies camera zoom by <code>+zoomStep</code>.
How to use in code	• Call <code>_zoomGestureDetector.detect(...)</code> after custom/recording checks and before movement. • Start pose: thumb-index distance ratio is closed. • Movement: open thumb/index enough within max gesture duration. • If <code>result == ZoomDirection.zoomIn</code>, call <code>_handleZoomDirection(result)</code>.
Points used	• Active pinch points: <code>thumbTip</code> and <code>indexFingerTip</code> (ids 4, 8). • Middle/ring/pinky folded check uses their MCP/PIP/tip points. • Palm center and hand bounding box define <code>handSize</code> and active tip-center distance.
Angles and thresholds	• Middle, ring, pinky folded by angle: ≤ 145 deg. • Thumb/index distance ratio = $\text{dist}(\text{thumbTip}, \text{indexTip}) / \text{handSize}$. • Valid active tips: center of <code>thumbTip/indexTip</code> must be farther than $\text{handSize} * 0.06$ from palm center. • Closed start ratio: ≤ 0.26 and held for 500 ms. • Open finish ratio: ≥ 0.27 and increased by at least 0.035. • Gesture stable duration: ≥ 90 ms; max gesture duration: 2600 ms. • Detected result is held for 650 ms. Camera zoom step is +0.2.
Overlap priority	• Zoom is evaluated before direction movement, and active zoom blocks Move Up/Down/Left/Right.

13. Zoom Out

Start with thumb and index open, hold briefly, then pinch them together.

Code path	• <code>ZoomGestureDetector.detect(...)</code> state machine. • <code>_handleZoomDirection(ZoomDirection.zoomOut)</code> applies camera zoom by <code>-zoomStep</code>.
How to use in code	• Call <code>_zoomGestureDetector.detect(...)</code> while no higher-priority custom/recording gesture is active. • Start pose: thumb-index distance ratio is open. • Movement: close thumb/index enough within max gesture duration. • If <code>result == ZoomDirection.zoomOut</code>, call <code>_handleZoomDirection(result)</code>.
Points used	• Active pinch points: <code>thumbTip</code> and <code>indexFingerTip</code> (ids 4, 8). • Middle/ring/pinky folded check uses their MCP/PIP/tip points. • Partial zoom-out recovery can use only <code>thumbTip/indexTip</code> divided by image max side.
Angles and thresholds	• Open start ratio: ≥ 0.27 and held for 500 ms. • Closed finish ratio: ≤ 0.26 and decreased by at least 0.035. • Gesture stable duration: ≥ 90 ms; max gesture duration: 2600 ms. • Partial zoom-out open image ratio: ≥ 0.045. • Partial close change: ≥ 0.018 and <code>distance</code> $\leq \text{startDistance} * 0.72$. • Detected result is held for 650 ms. Camera zoom step is -0.2.
Overlap priority	• Zoom out uses the same priority protection as Zoom In and blocks generic movement while active.